# LAB ASSIGNMENT-2

TEAM MEMBERS:

1. Anuj Saha (202351010)
2. Divyanshu Ghosh (202351036)
3. G Nikhil (202351037)

The solution to the given problem is addressed using affine transformation techniques in digital image processing. Affine transformation modifies the spatial arrangement of pixels in an image using a combination of linear transformations and translation.
The different functional components involved in the implementation are described below.

- **Affine Transformation Module**
  - Accepts an input image selected by the user.
  - Allows the user to specify affine transformation parameters such as:
    i. Horizontal scaling factor (more than 0)
    ii. Vertical scaling factor (more than 0)
    iii. Rotation angle (ranging from -180 to +180 degrees)
    iv. Horizontal translation (Any integer (in pixels))
    v. Vertical translation (Any integer (in pixels))
    vi. Horizontal shear factor (ranging from -1 to +1)
    vii. Vertical shear factor (ranging from -1 to +1)
  - Constructs individual transformation matrices for scaling, rotation, shearing, and translation.
  - Combines all transformations into a single composite affine transformation matrix using homogeneous coordinates.

## How to Run the Program

1. Place `Affine_Transformation.m` and an input image in the same folder.
2. Run the `Affine_Transformation.m`.
3. When prompted:
   - Select the image.
   - Enter the required affine transformation parameters.

4. The affine transformation is applied to the image.
5. The original image and the transformed image are displayed.

---

# Affine_Transformation.m

```
clc;
clear;
close all;
%% Taking the Image as input
[file, path] = uigetfile({'*.jpg;*.jpeg;*.png;*.bmp'}, 'Select an Image');
if isequal(file,0)
   disp('No image selected');
   return;
end
%% Grayscale conversion
img = imread(fullfile(path, file));
if ndims(img) == 3
   R = img(:,:,1);
   G = img(:,:,2);
   B = img(:,:,3);
   img = 0.2989 * double(R) + 0.5870 * double(G) + 0.1140 * double(B);
else
   img = double(img);
end
[H, W] = size(img);
%% inputs for the parameters
% Parameter Ranges
% Rotation (theta): -180 to +180 degrees [Positive -> anticlockwise
% rotation, Negative -> clockwise rotation]%
% Scaling factors: [Values < 1 shrink the image, Values = 1 keep original
% size, Values > 1 enlarge the image]
%   sx (horizontal scaling) : sx > 0
%   sy (vertical scaling)   : sy > 0
% Translation parameters (in pixels):
%   tx (horizontal translation) : any integer
%   ty (vertical translation)   : any integer
%   [Positive tx -> right, Negative tx -> left, Positive ty -> down,
%   Negative ty -> up]
% Shear factors: [Larger absolute values cause stronger distortion]
%   shx (horizontal shear) : -1 to +1 (practical range)
%   shy (vertical shear)   : -1 to +1 (practical range)
sx  = input('Horizontal scaling factor: ');
sy  = input('Vertical scaling factor: ');
theta = input('Rotation angle (degrees): ');
tx  = input('Horizontal translation: ');
ty  = input('Vertical translation: ');
```

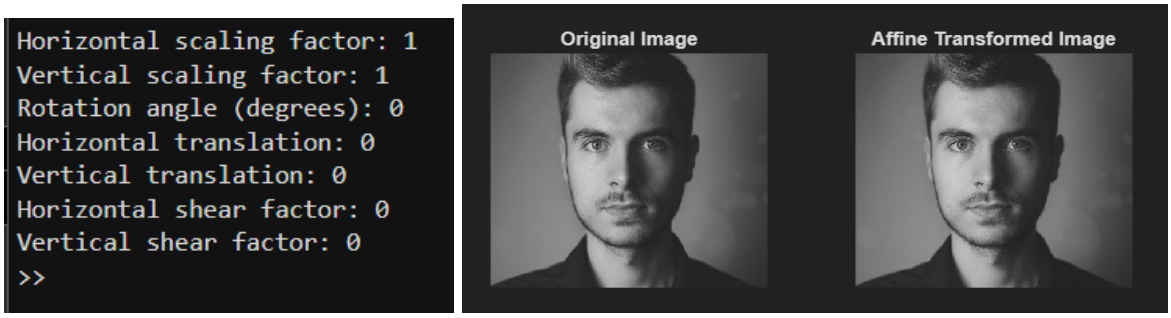
```
shx = input('Horizontal shear factor: ');
shy = input('Vertical shear factor: ');
theta = deg2rad(theta);
%% validating parameter inputs
% Rotation angle
if theta < -180 || theta > 180
   error('Rotation angle must be between -180 and +180 degrees');
end
% Scaling factors
if sx <= 0 || sy <= 0
   error('Scaling factors sx and sy must be greater than 0');
end
% Translation (optional limits for lab safety)
if abs(tx) > max(W, H) || abs(ty) > max(W, H)
   error('Translation values are too large for the image size');
end
% Shear factors
if abs(shx) > 1 || abs(shy) > 1
   error('Shear factors must be between -1 and +1');
end
%% matrices for transformation
S = [ sx   0   0
     0   sy   0
     0    0   1 ];
Sh = [ 1   shx   0
     shy   1    0
      0    0    1 ];
R = [ cos(theta)  -sin(theta)   0
     sin(theta)   cos(theta)   0
          0            0       1 ];
T = [ 1   0   tx
     0   1   ty
     0   0    1 ];
M = T * R * Sh * S; %% composite matrix
M_inv = inv(M); %% inverse matrix
output = zeros(H, W); %% output image
%% affine transformation applied
for y = 1:H
   for x = 1:W
       src = M_inv * [x; y; 1];
       src_x = round(src(1));
       src_y = round(src(2));
       if src_x >= 1 && src_x <= W && src_y >= 1 && src_y <= H
           output(y, x) = img(src_y, src_x);
       end
   end
end
%% displayed result
figure;
subplot(1,2,1);
```

```
imshow(uint8(img));
title('Original Image');
subplot(1,2,2);
imshow(uint8(output));
title('Affine Transformed Image');
```

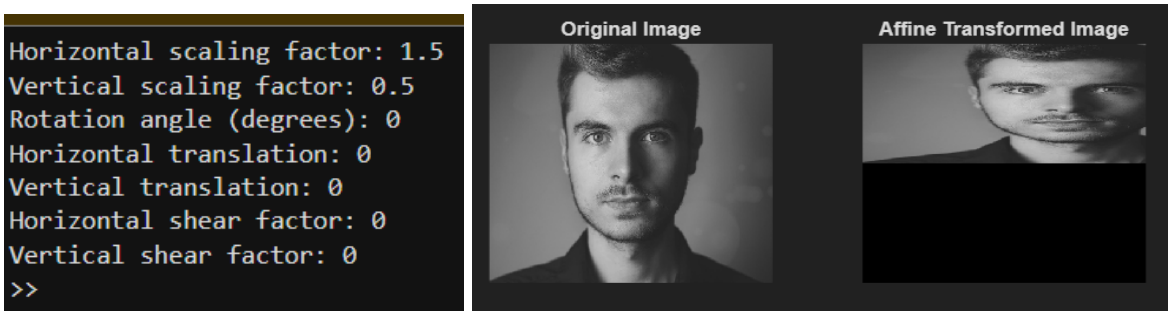
---

# Results

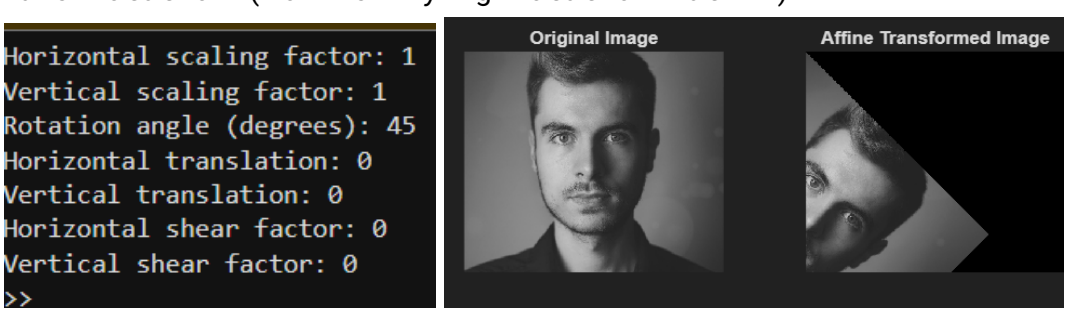
1. Identity Transformation (for verifying code correctness)




2. Pure Scaling (for checking scaling logic)




3. Pure Rotation (for verifying rotation matrix)

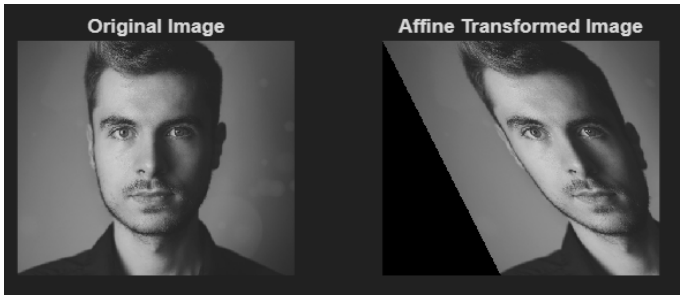


4. Pure Translation (for checking translation handling)

```
Horizontal scaling factor: 1
Vertical scaling factor: 1
Rotation angle (degrees): 0
Horizontal translation: 50
Vertical translation: 30
Horizontal shear factor: 0
Vertical shear factor: 0
```

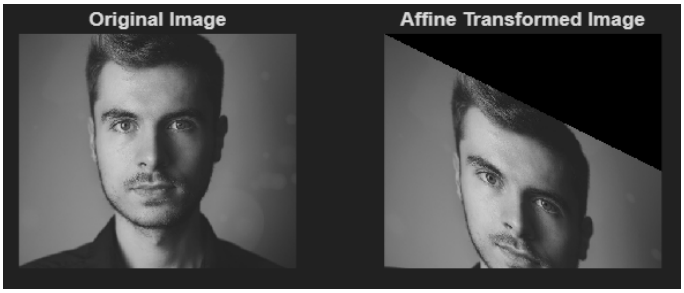
5. Horizontal Shear (for validating shear matrix)

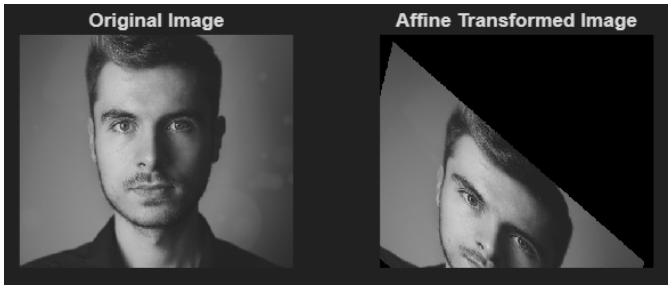
```
Horizontal scaling factor: 1
Vertical scaling factor: 1
Rotation angle (degrees): 0
Horizontal translation: 0
Vertical translation: 0
Horizontal shear factor: 0.5
Vertical shear factor: 0
```




6. Vertical Shear

```
Horizontal scaling factor: 1
Vertical scaling factor: 1
Rotation angle (degrees): 0
Horizontal translation: 0
Vertical translation: 0
Horizontal shear factor: 0
Vertical shear factor: 0.5
```




7. Complete Affine Transformation (verifying everything works together)

```
Horizontal scaling factor: 1.2
Vertical scaling factor: 0.8
Rotation angle (degrees): 30
Horizontal translation: 40
Vertical translation: 20
Horizontal shear factor: 0.3
Vertical shear factor: 0.2
```